

REPORT

Exercise 3

1.

Applying regex on a single line will not work because comments in Jack can span multiple lines. A tokeniser that reads a file one line at a time will only be able to match the regex pattern when the comment spans a single line. It will not be able to match the comment correctly when there are line breaks in the comment as the opening `/*` and closing `*/` will not be found on the same line.

Failure case:

```
/* This program computes the product of two integers.
```

```
We take the inputs from the user.
```

```
The output is displayed on the screen */
```

The tokenizer sees `/* This program computes the product of two integers,` does not see a closing `*/`, and incorrectly tokenizes the text or returns an error. Using a state machine allows the tokenizer to anticipate multiple lined comments on matching `/*`.

2.

Class level symbol table:

Identifier	kind	type	index
ArrayFilter	static	Array	0
stride	static	int	1
ArrayInput	field	Array	0
ArrayOutput	field	Array	1
ArraySize	field	int	2

Subroutine level symbol table (conv2d):

Identifier	kind	type	index
this	argument	Conv	0
index	local	int	0
i	local	int	1

j	local	int	2
ans	local	int	3
ii	local	int	4
jj	local	int	5
id	local	int	6

3.

When the Jack compiler sees `let ArrayOutput[index] = ans;` it calculates the destination RAM address, the value to store, and then stores the value into the destination.

Code:

```

push this 1    // base address of ArrayOutput
push local 0   // value of index local variable
add          // offsetting the ArrayOutput base address with + index
push local 3   // value of ans local variable
pop temp 0     // store ans local variable value in RAM[5]
pop pointer 1  // store target address in that pointer
push temp 0    // value of ans local variable
pop that 0     // store ans value at memory address pointed to by that

```

Pointer 1 refers to the 'that' register (location stored at RAM[4]). When we call `pop pointer 1`, the 'that' register points to the calculated target address. When we call `pop that 0`, the VM takes the ans value and stores it in RAM[that]. This dereferences the pointer and writes the value into the target array location.

4.

$$O = [(N - F) / S] + 1$$

Given $N = 9$, $F = 3$, and $S = 2$:

Output matrix dimensions = $4 * 4$

Changing $S = 1$ to $S = 2$ does not change the asymptotic vm instruction count. The number of instructions executed scales proportionally to N^2 / S^2 . Changing the S from 1 to 2 reduces the number of loop iterations by a factor of 4, but in asymptotic notation, this is irrelevant. The asymptotic complexity of the method remains $O(N^2)$.

Convolution operation logs

Note: I asked gemini to generate an example which would be easy to verify

3*3 filter

0 0 0

0 1 0

0 0 0

(output's the matrix without outer-most row and column when stride = 1 else alternates entries when stride = 2)

5*5 matrix

0 1 2 3 4

5 6 7 8 9

10 11 12 13 14

15 16 17 18 19

20 21 22 23 24

9*9 matrix

0 1 2 3 4 5 6 7 8

9 10 11 12 13 14 15 16 17

18 19 20 21 22 23 24 25 26

27 28 29 30 31 32 33 34 35

36 37 38 39 40 41 42 43 44

45 46 47 48 49 50 52 52 53

54 55 56 57 58 59 60 61 62

63 64 65 66 67 68 69 70 71

72 73 74 75 76 77 78 79 80

Screen

x0

x1

x2

```
5x5 input with stride 1
6 7 8
11 12 13
16 17 18
```

```
9x9 input with stride 2
10 12 14 16
28 30 32 34
46 48 50 52
64 66 68 70
```

Extra Credit

While implementing the SymbolTable, I initially reset the entire counts dictionary inside startSubroutine. For argument and var this is correct as those counters must restart at 0 for each new subroutine. But resetting static and field variables as well led to them returning incorrect counts and being overwritten. The fix was changing startSubroutine to only reset the subroutine-level counters and leave the class-level ones untouched.

Note: To test the translated .vm files from Conv.jack and Main.jack I have used the VM Emulator instead of the CPU emulator. The logs I pasted were from the same. To test compatibility with the VM translator built in Assignment 2, I downloaded .vm files of the OS tools and translated them to .asm using the VM translator built in Assignment 2. Unfortunately, I was not able to test the performance on the CPU emulator as the IDE was frequently crashing and/or was very slow (considering the huge size of the .asm file).